Module 1: Introduction to Artificial Neural Networks (ANNs)

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Getting Ready for the Course

Knowledge Prerequisites

- **Machine Learning:** Solid understanding of **Linear & Logistic Regression**, and Multiclass Classification.
- **Programming:** Proficiency in **Python**, including libraries like **NumPy**, Pandas, and **Matplotlib**.
- **Mathematics:** Basic knowledge of **Calculus (Chain Rule)** and **Linear Algebra (Vectors, Matrices)**.

Setting Up Your Environment

- **Essential Libraries:**
  - The core frameworks: **PyTorch** or **TensorFlow**.
  - Scientific computing and plotting: **NumPy** & **Matplotlib**.
- **Development Environment:**
  - Highly recommended: **Jupyter Notebooks** or **Google Colab**.

Outline of this Module

1. From Machine Learning to Deep Learning
2. The Anatomy of a Neural Network
3. The Learning Loop: How Neural Networks Learn

Machine Learning Landscape

From Traditional Machine Learning to Deep Learning

Machine Learning

Arthur Samuel (1959)

Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.

Learn directly from data - No explicitly programmed rules

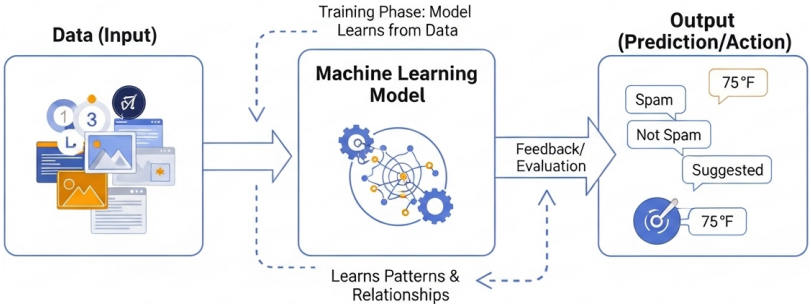


Figure: ML: Learning from Data (Source: ImageGen)

Common Machine Learning Applications

Figure: Housing Price Prediction

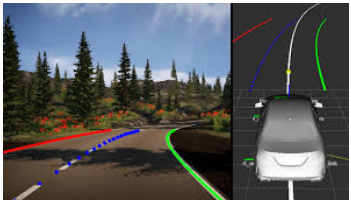


Figure: Autonomous vehicles(Lane following)

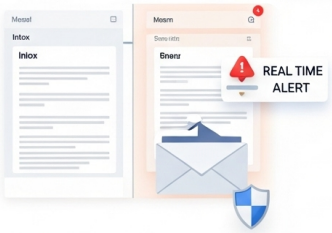


Figure: Message Sentiment Analysis

Common Machine Learning Applications

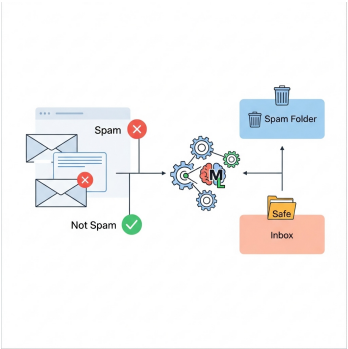


Figure: Spam Email Filtering

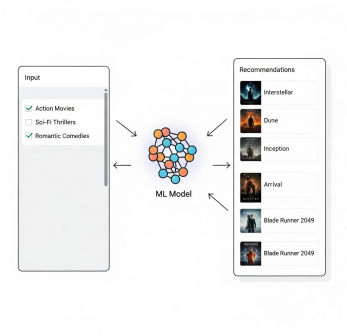


Figure: Recommendation engines

Figure: Image Recognition

Machine Learning: Key Concepts

The Data & Process

Focus: Structured Data

- Data in tables (rows & columns).
- Predict a target from features.

Method: Manual Feature Engineering

- Human experts design features.
- Model learns from these engineered features.
- Success depends entirely on feature quality.

The Models

Linear Models

- *Core Idea:* Assumes linear relationships.
- *Examples:* Linear/Logistic Regression.
- *Traits:* Fast, interpretable, but simple.

Tree-Based Models

- *Core Idea:* Learns ‘if-then-else‘ rules.
- *Examples:* Random Forest, Gradient Boosting.
- *Traits:* Captures non-linearity, powerful.

Limitations of Traditional Machine Learning

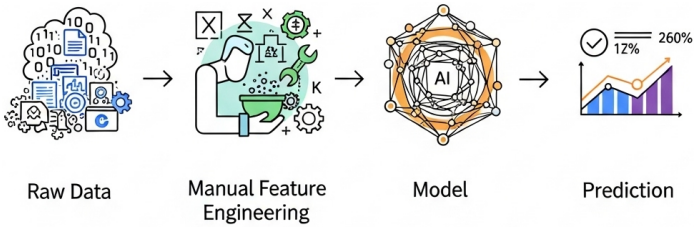


Figure: Feature Engineering Bottleneck (Source: ImageGen)

- **The Feature Engineering Bottleneck**: Time-consuming, requires domain expertise, and is often suboptimal.
- **Inability to handle high-dimensional data**: Struggles with images, audio, or raw text.
- **Limited representation learning**: Learns shallow patterns, not deep, hierarchical features.
- **Performance plateaus**: Adding more data yields diminishing returns after a certain point.

The Deep Learning Revolution

From Traditional Machine Learning to Deep Learning

The Shift to Deep Learning: A New Paradigm

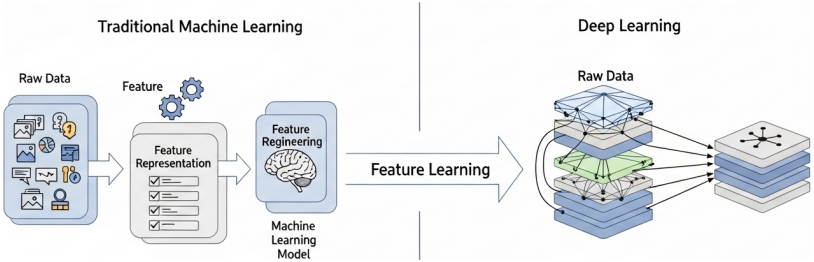


Figure: ML needs manual feature extraction; DL learns them automatically.

Deep Learning: The Core Idea

- Models learn directly from raw data.
- Handles images, text, and audio.
- **No manual feature engineering.**

Why this Paradigm Shift?

- Solves the feature engineering bottleneck.
- Automates representation learning.
- Unlocks new, complex capabilities.

Popular Deep Learning Applications

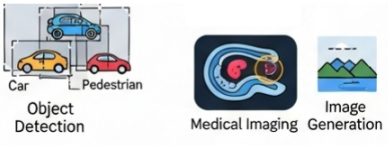


Figure: Computer vision

Figure: Natural language processing

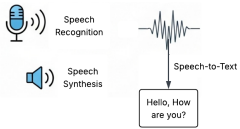


Figure: Speech recognition and Audio synthesis

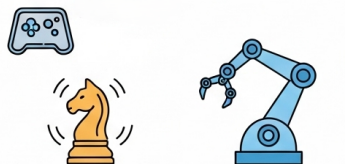


Figure: Game playing and robotics

Key Differences: ML vs. DL

Machine Learning

- **Feature Extraction:**
  Requires manual engineering.
- **Representation:**
  Learns shallow patterns.
- **Data Scalability:**
  Works well on small to medium data.

Deep Learning

- **Feature Extraction:**
  Learns features automatically.
- **Representation:**
  Builds hierarchical features.
- **Data Scalability:**
  Excels with very large datasets.
- **Training Paradigm:**
  Enables end-to-end learning.

What Enabled the Deep Learning Revolution?

A convergence of three key factors created the "perfect storm"

Big Data

- Massive labeled datasets.
- The "fuel" for hungry models.
- e.g., ImageNet, Wikipedia.

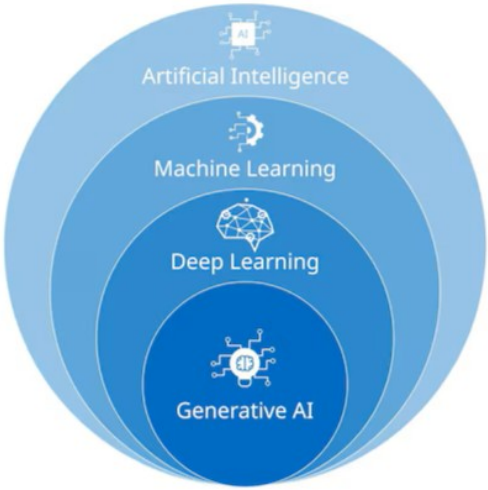
Computational Power

- Rise of GPUs (parallel power).
- Made deep training feasible.
- Drastically cut training time.

Better Algorithms

- Architectural innovations.
- e.g., CNNs, Transformers.
- Smarter training methods.

The AI Venn Digram



From Biology to Mathematics

The Anatomy of a Neural Network

Inspiration from the Brain

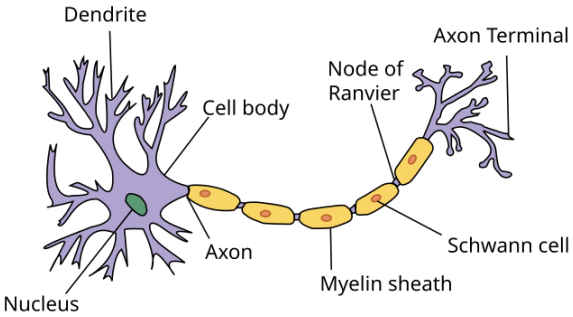


Figure: A biological neuron (Source: Wikipedia)

A Simple Model of a Neuron:

- Receives signals via Dendrites.
- Integrates signals in the Soma/Cell body.
- If threshold is met, it "Fires".
- Sends output signal via Axon.

Important: Inspiration, Not a Replica

- Brain is vastly more complex.
- Biological learning is not backpropagation.
- This is a **useful abstraction**, not a perfect model.

The McCulloch-Pitts Neuron (1943)

The Core Idea: A Simple Biological Logic Gate

The first mathematical model of a neuron. It operates on a simple principle: if enough inputs are active, the neuron "fires."

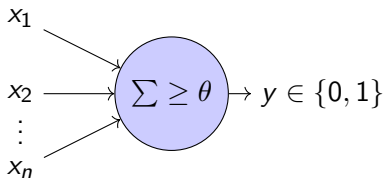


Figure: The M-P Neuron Model: Sum inputs and compare to a threshold θ.

How it Works:

- Takes multiple binary inputs ($x_i \in \{0, 1\}$).
- Calculates their sum.
- The output is 1 if the sum is greater than or equal to a fixed threshold (θ).
- The output is 0 otherwise.

Significance:

- Proved that networks of these simple units could compute any logical function (e.g., AND, OR).

Critical Limitations of the M-P Neuron

A Calculator, Not a Learner

The M-P neuron was a fixed logic gate. It could compute, but it could not **learn**.

No Feature Importance

- Equally important features.
- Can't prioritize key features.
- *Missing piece: Weights.*

No Automatic Learning

- Threshold (θ) is fixed.
- Must be set manually.
- *Missing piece: A Learning Rule.*

Restricted to Binary I/O

- Inputs must be 0 or 1.
- Outputs are only 0 or 1.
- Can't handle real-valued data.

The Leap to Learning: The Perceptron (1958)

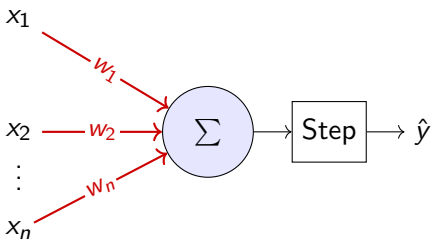


Figure: The Perceptron: Introduces learnable weights (w_i) on each input.

Innovation 1: Weighted Inputs

- Each input is multiplied by a weight.
- Weights represent feature importance.
- The model can now **prioritize** signals.

Innovation 2: The Learning Rule

- Compares prediction ($\hat{y}$) to true label (y).
- If wrong, it adjusts the weights to correct the error.

Significance: The Birth of Learning Machines

For the first time, **a machine could automatically learn** to classify patterns, laying the foundation for all of modern deep learning.

Perceptron: Strengths & Limitations

Strengths

- **It Learns!**
  An automatic learning rule adjusts weights from data.
- **Feature Importance:**
  Learns which inputs are more important via weights.
- **Convergence Guarantee:**
  If data is linearly separable, it's guaranteed to find a solution.

Limitations

- **The Fatal Flaw: Linear Separability** Only problems where a single straight line (or hyperplane) can separate the classes can be solved.
- **Harsh Threshold:** The step function activation is not differentiable, preventing modern gradient-based training.

The Modern Neuron: Engine of Deep Learning

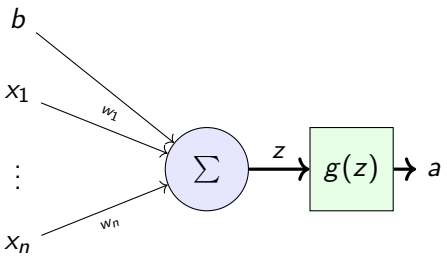


Figure: A neuron with differentiable activation function

Step 1: Linear Combination

- Calculates a weighted sum of inputs plus a bias.
- Output is $z = \mathbf{w} \cdot \mathbf{x} + b$.

Step 2: Non-Linear Activation

- Passes the linear output z through an differentiable activation function $a = g(z)$.

Why Differentiability is the Superpower?

Differentiable $\rightarrow$ **gradient**. This gradient is used to adjust weights to fix errors. It is the core requirement for **backpropagation**.

Fundamental Tricks in Deep Learning

The Anatomy of a Neural Network

The Numerization Trick: From Sensation to Numbers

The Core Problem: Computers Don't See, They Calculate

- Humans perceive the world through qualitative sensations.
- Machines only understand quantitative data (numbers).

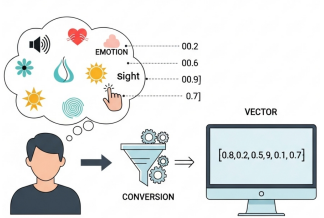


Figure: Qualitative representation to Quantitative representation

Our Goal

Convert every piece of information about a problem into a list of numbers, called a **vector**.

The Numerization Trick: Example

Scenario: A Self-Driving Car

A car's AI must perceive the road, identify objects, and decide how to act in real-time. **Task:** How do we represent its world as Input and Output vectors?

Input Vectors (What the Car "Sees")

- **Camera Data:** A massive vector of pixel values (R,G,B for each pixel).
- **LiDAR Data:** A vector of 3D point coordinates (x, y, z) for nearby surfaces.
- **Radar Data:** A vector of object distances and velocities.
- **Car's State:** A vector of current speed, acceleration, GPS location.

Output Vectors (What the Car "Does")

- **Control Actions:** A vector like '[Steering Angle, Acceleration, Brake]'.
- **Object Predictions:** For each detected object, a vector like '[Class, X-Pos, Y-Pos, Velocity]'.

How We Numerize: Encoding Categories

The Challenge

How do we convert an abstract category like "Pedestrian" or "Stop Sign" from the car's output vector into numbers?

Method 1: One-Hot Encoding

- Represents a class as a binary vector.
- One position is "hot" (1), all others are 0.
- **Car Example (Object Class):**
  - Pedestrian $\rightarrow$ `[1, 0, 0]`
  - Vehicle $\rightarrow$ `[0, 1, 0]`
  - Stop Sign $\rightarrow$ `[0, 0, 1]`
- Simple but inefficient for many classes.

Method 2: Embedding Layers

- Learns a dense, meaningful vector for each class.
- Similar concepts get similar vectors.
- **Car Example (Object Class):**
  - Stop Sign $\rightarrow$ `[0.9, -0.2, ...]`
  - Pedestrian $\rightarrow$ `[-0.5, 0.7, ...]`
- Efficient & captures semantic relationships.

Learn the Function Trick

Data and Functions

- We frame problems as data (inputs & outputs).
- We frame solutions as a **function** that maps inputs to outputs.
- The "learning task" is to find this function.

Every Function Has Two Parts

- **The Form (Architecture):**
  - The function's template or structure.
  - Learning a structured function is more tractable than learning an arbitrary function.
- **The Parameters (w):**
  - The "dials" that tune the function.
  - Learned automatically from data.

E.g., ChatGPT can be seen as one, massive function taking your prompt as input.

The Parameterization Trick

Core Idea: We define our model as a function with a set of learnable parameters (weights and biases), denoted by θ. The goal of training is to find the optimal values for θ.

The Model as a Function

We express the model's prediction as

$$\hat{y} = f(x; \theta),$$

where:

- x is the numerical input data.
- f is the network architecture (e.g., layers, activation functions).
- θ represents all the weights and biases in the network.
- $\hat{y}$ is the model's prediction.

From Learning to Optimization: The Loss Landscape

Parameterization turns "learning" into a clear problem: optimization.

Step 1: Measuring Error

- A **Loss Function**, $\mathcal{L}(y, \hat{y}) = \mathcal{L}(y, f(x; \theta)) \approx \mathcal{L}(\theta)$, scores how "bad" our model is for a given set of parameters θ.
- High Loss = Bad Model.
  Low Loss = Good Model.

Step 2: The Goal

- Our goal is to find the parameters θ^* that result in the lowest possible loss.

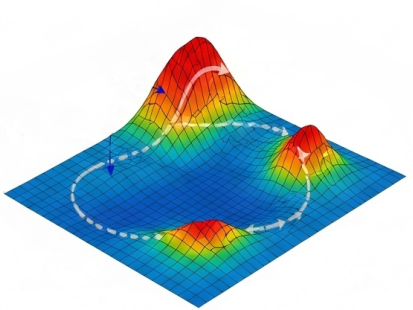


Figure: This creates a high-dimensional **Loss Landscape** with mountains (high error) and valleys (low error).

Feedforward Network Architecture

The Anatomy of a Neural Network

Network Topology: Building a Feedforward Network

1. The Input Layer

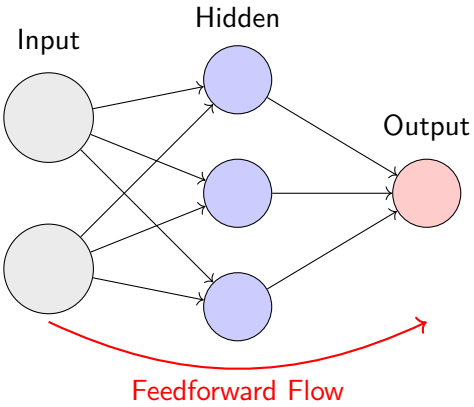
- Receives the data and just pass values along.

2. The Hidden Layers

- The network's computational engine.
- They learn increasingly abstract features.

3. The Output Layer

- Produces the final result (e.g., a prediction).



- **Feedforward Flow:** Information moves in one direction only: input → output. No loops.
- **Fully Connected:** Every neuron connects to every neuron in the *next* layer.

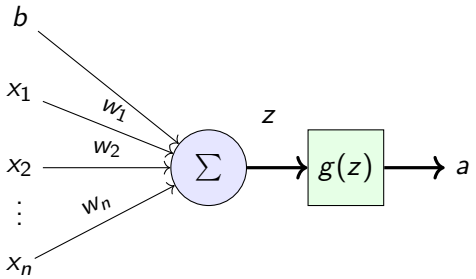
The Standard Forward Model: A Neuron's Calculation

Step 1: The Linear Part (Weighted Sum)

- A weighted sum of all inputs is calculated.
- A bias term is added to this sum.
- **Result:** $z = (\sum_i w_i x_i) + b$

Step 2: The Non-Linear Part (Activation)

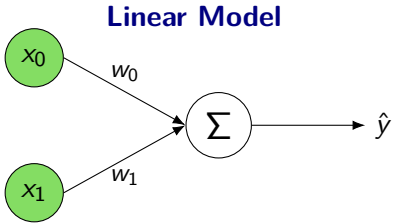
- The result z is passed through a non-linear activation function g.
- This introduces complexity and allows the network to learn non-linear patterns.
- **Final Output:** $a = g(z)$

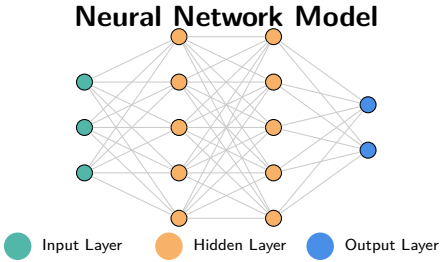


The Fundamental Building Block

This two-step 'Linear Sum → Non-Linear Activation' process is performed by every single neuron in a deep neural network.

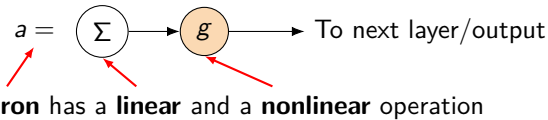
Linear Model vs Artificial Neural Networks





$$\hat{y} = w_0 x_0 + w_1 x_1 = \sum w_i x_i$$

$$a = g\left(\sum w_i x_i\right)$$



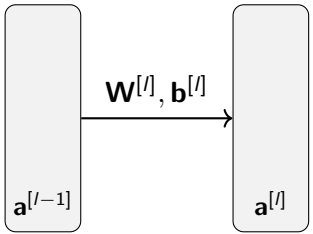
A Linear Model is weighted sum of input features.

A **Neuron** has a **linear** and a **nonlinear** operation

The Forward Pass: The Vectorized Layer View

The Problem & The Solution

- **Problem:** Calculating neuron by neuron is incredibly inefficient.
- **Solution:** Compute an entire layer at once using vectorized operations (i.e., matrix multiplication).



Let's define the terms for a layer l:

- $\mathbf{a}^{[l-1]}$: Vector of activations from the previous layer.
- $\mathbf{W}^{[l]}$: Weight matrix, where W_{ij} is the weight from neuron j in layer $l-1$ to neuron i in layer l.
- $\mathbf{b}^{[l]}$: Bias vector for the current layer.
- $\mathbf{z}^{[l]}$: Vector of weighted sums for the current layer.
- $\mathbf{a}^{[l]}$: Vector of activations for the current layer.

The Forward Pass: The Vectorized Layer View

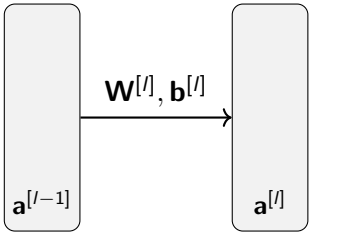


Figure: A layer's computation maps an input vector to an output vector.

The Two Core Equations (Vectorized)

1. **Weighted Sum:** $\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$
2. **Activation:** $\mathbf{a}^{[l]} = g(\mathbf{z}^{[l]})$

This is computationally efficient and perfectly suited for GPUs.

Let's compute the output of a hidden layer with 3 neurons, receiving input from 2 neurons.

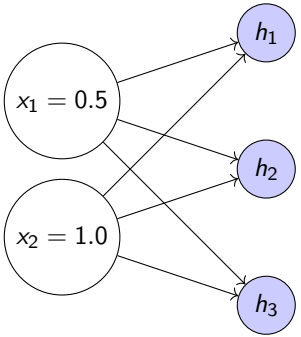


Figure: Input vector $\mathbf{x}$ has 2 features. Hidden layer has 3 neurons.

Let the activation function be ReLU

$$g(z) = \max(0, z)$$

1. Define inputs, weights, and biases:

$$\mathbf{x} = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 0.2 & 0.7 \\ -0.4 & 0.1 \\ 0.9 & -0.3 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 0.1 \\ 0.2 \\ -0.5 \end{bmatrix}$$

2. Calculate the weighted sum $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$:

$$\mathbf{z} = \begin{bmatrix} 0.2 & 0.7 \\ -0.4 & 0.1 \\ 0.9 & -0.3 \end{bmatrix} \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0.9 \\ 0.1 \\ -0.35 \end{bmatrix}$$

3. Apply element-wise activation $\mathbf{a} = \mathbf{ReLU}(\mathbf{z})$:

$$\mathbf{a} = \text{ReLU}\left(\begin{bmatrix} 0.9 \\ 0.1 \\ -0.35 \end{bmatrix} \right) = \begin{bmatrix} \max(0, 0.9) \\ \max(0, 0.1) \\ \max(0, -0.35) \end{bmatrix} = \begin{bmatrix} 0.9 \\ 0.1 \\ 0 \end{bmatrix}$$

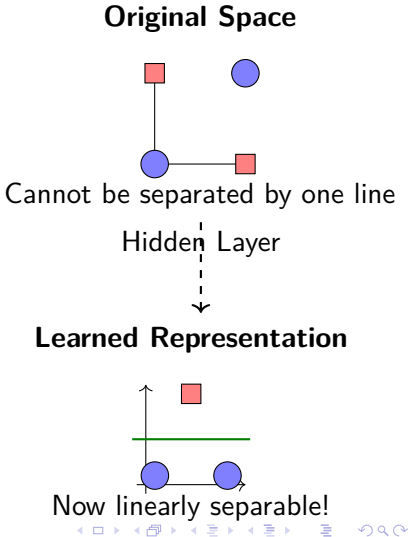
The Multilayer Perceptron (MLP): Overcoming Limitations

What is an MLP?

Simply a feedforward network with **one or more hidden layers**.

The Power of the Hidden Layer

- A single Perceptron fails on non-linear data (like XOR).
- The hidden layer acts as an automatic feature engineer.
- It learns to **transform the data** into a new representation.
- In this new space, the data becomes linearly separable.
- This allows the MLP to create complex decision boundaries.



Activation Functions: Introducing Non-Linearity

The Anatomy of a Neural Network

Why Do We Need Activation Functions?

The Problem: Stacking Linear Layers is Useless

Without a non-linear activation function, a deep network simply collapses into a single linear model, no matter how many layers it has.

Without Non-Linearity:

- A layer is a linear operation: $\mathbf{W}\mathbf{x} + \mathbf{b}$.
- Stacking them: $L_2(L_1(\mathbf{x})) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$.
- This simplifies to: $(\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$.
- This is just another linear function: $\mathbf{W}'\mathbf{x} + \mathbf{b}'$.

A 100-layer linear network has the same power as a 1-layer network.

With Non-Linearity (g):

- The equation becomes: $g(\mathbf{W}_2 g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$.
- This function **cannot** be simplified.
- This allows the network to learn arbitrarily complex, "wiggly" functions.

Conventional Activations: Sigmoid

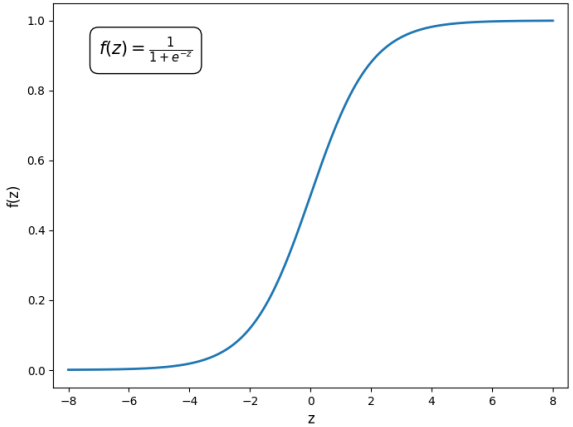


Figure: Sigmoid Function

Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties

- Squeezes any real number into the range $(0, 1)$.
- Historically popular for its interpretation as a neuron's "firing rate."
- Useful in output layers for binary classification (predicting probabilities).

Conventional Activations: Sigmoid

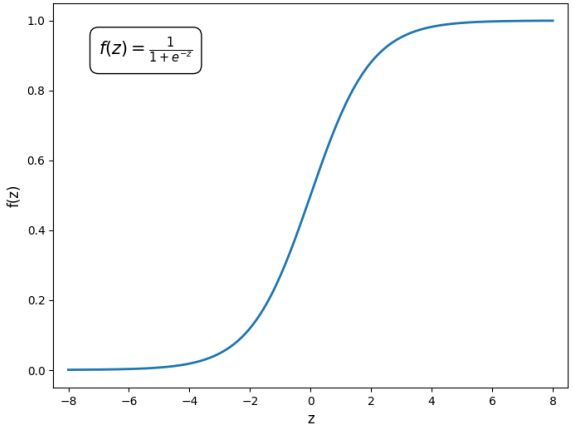


Figure: Sigmoid Function

Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Problems

- **Vanishing Gradients:** The function is flat at both ends. The gradient is near zero for large positive or negative inputs, which effectively stops learning in deep networks.
- **Not Zero-Centered:** Outputs are always positive, which can slow down learning.

Conventional Activations: Tanh

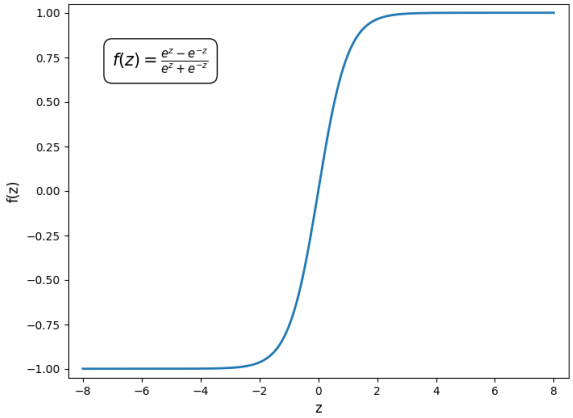


Figure: Tanh Function

Formula:

$$tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Properties

- Squeezes any real number into the range $(-1, 1)$.
- **Zero-Centered:** Its major advantage over Sigmoid. This property helps center the data for the next layer, often speeding up convergence.

Conventional Activations: Tanh

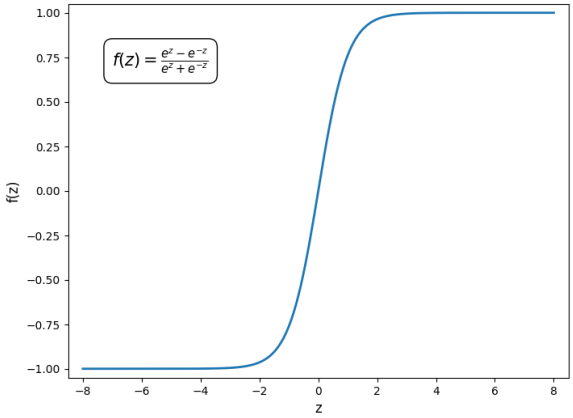


Figure: Tanh Function

Formula:

$$tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Problems

- **Vanishing Gradients:** Like Sigmoid, the function flattens out, and its gradients disappear for large positive or negative inputs, making it difficult to use in very deep networks.

Conventional Activations: ReLU

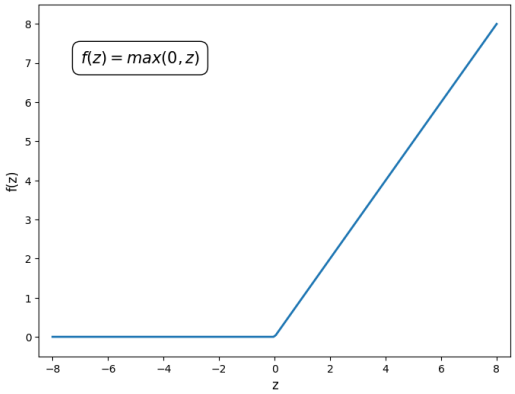


Figure: ReLU Function

Formula:

$$\text{ReLU}(z) = \max(0, z)$$

Properties

- **Computationally Efficient:** Very fast to compute (just a threshold).
- **No Vanishing Gradient (for $z > 0$):** The gradient is a constant 1 for positive inputs, allowing learning signals to propagate deep into the network.
- **Sparsity:** By outputting 0 for negative inputs, it can make the network sparse.

Conventional Activations: ReLU

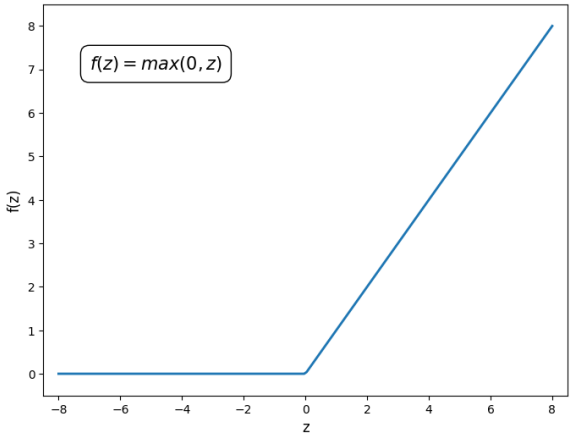


Figure: ReLU Function

Formula:

$$\text{ReLU}(z) = \max(0, z)$$

Problems

- **The "Dying ReLU" Problem:** If a neuron's weights are updated such that its pre-activation z is always negative, it will always output 0. The gradient will also be 0, and the neuron can never recover. It effectively "dies."

The ReLU Family: Fixing the "Dying" Problem

Leaky ReLU

$$f(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}$$

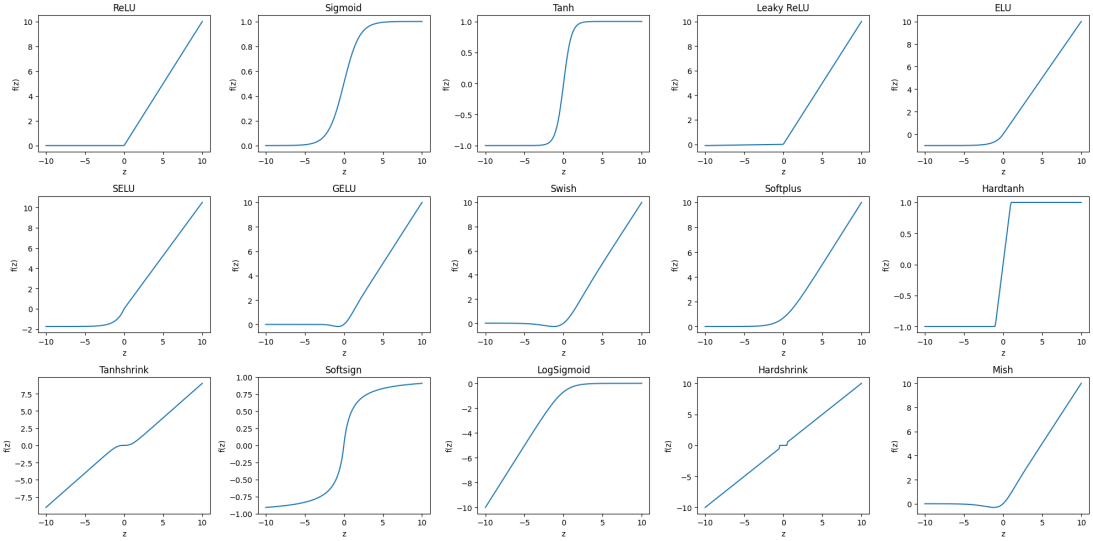
(where α is a small constant like 0.01)

- **The Fix:** Introduces a small, non-zero gradient for negative inputs.
- This allows "dead" neurons to be revived. A very common and effective choice.

GELU (Gaussian Error Linear Unit)

- A smoother, probabilistic alternative to ReLU.
- Became the standard in state-of-the-art **Transformer** models (e.g., GPT, BERT).

Popular Activation Functions



A Practical Guide to Choosing Activations

For Hidden Layers

1. **Start with ReLU.** Most common, fastest, and usually works.
2. For issues with "dying" neurons, switch to **Leaky ReLU** or **Parameteric ReLU**.
3. For Transformer-based models, consider **GELU** or **Swish**.

For the Output Layer (Task-Dependent)

Depends on the problem's output range.

- **Binary Classification: Sigmoid**. Outputs a probability between 0 and 1.
- **Multiclass Classification: Softmax**. Outputs a probability distribution over all classes (all outputs sum to 1).
- **Regression: None (Linear)**. The output can be any real number.

Rule of Thumb

Never use Sigmoid or Tanh in hidden layers of modern deep networks due to the vanishing gradient problem. Their use is now almost exclusively in output layers.

The Forward Pass and Measuring Error

The Learning Loop: How Neural Networks Learn

The Forward Pass: From Input to Prediction

The forward pass is the process of taking an input vector $\mathbf{x}$ and feeding it through the network, layer by layer, to produce a final prediction $\hat{\mathbf{y}}$.

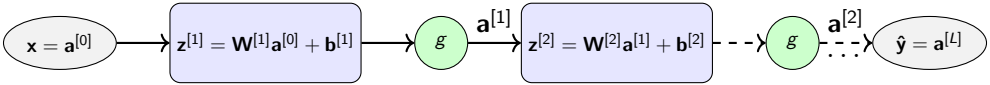


Figure: Information flows sequentially through linear layers and non-linear activations.

The computation for every layer follows the same two-step, vectorized pattern:

1. **Linear Combination:** Calculate the weighted sum for all neurons in the layer.

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

2. **Non-linear Activation:** Apply the activation function element-wise.

$$\mathbf{a}^{[l]} = g(\mathbf{z}^{[l]})$$

The final activation vector $\mathbf{a}^{[L]}$ is the network's prediction $\hat{\mathbf{y}}$.

Loss Functions: Quantifying the Error

A loss function $\mathcal{L}(y, \hat{y})$ takes the true label (y) and the prediction ($\hat{y}$) and returns a single number representing the error. This error is what we aim to minimize during training.

For Regression Tasks

Mean Squared Error (MSE)

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

- Calculates the average squared difference between true and predicted values.
- Penalizes large errors very heavily.

Loss Functions: Quantifying the Error

For Classification Tasks

Cross-Entropy Loss

$$\mathcal{L}_{CE} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

- y_c is 1 if c is the true class, 0 otherwise.
- $\hat{y}_c$ is the predicted probability for class c.
- Penalizes the model for being confident and wrong.

Example (3 classes):

- **True:** Cat $\rightarrow [1, 0, 0]$
- **Good Pred:** $[0.8, 0.1, 0.1] \rightarrow$ Low Loss
- **Bad Pred:** $[0.1, 0.2, 0.7] \rightarrow$ High Loss

Gradient Descent

The Learning Loop: How Neural Networks Learn

Gradient Descent: The Intuition

The Analogy: A Hiker on a Foggy Mountain

Imagine you are on a mountainside in thick fog and want to get to the lowest valley. What do you do?

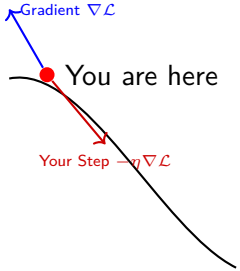
You ask two simple questions at every step:

1. Which way is downhill?

- You feel the slope under your feet.
- The direction of steepest ascent is the Gradient ($\nabla \mathcal{L}$).
- So, you take a step in the **opposite** direction.

2. How big a step should I take?

- This is your "step size" or Learning Rate (η).
- Too small: takes forever. Too big: overshoots the valley.



Why Gradient Descent? Navigating the Loss Landscape

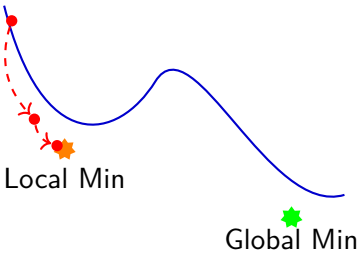
The Goal

Find the parameters θ that minimize the Loss Function $\mathcal{L}(y, f(x;\theta))$. This means finding the lowest point in the landscape.

The Update Rule

At each step, we move in the direction of steepest descent:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla \mathcal{L}(y, f(x;\theta_{old}))$$



The Challenge: Local vs. Global Minima

- **Global Minimum:** The true lowest point; the optimal solution.
- **Local Minimum:** A "valley" that traps the algorithm, preventing it from reaching the global minimum.

GD in Action: Linear Regression

1. The Model

Find the best line $\hat{y} = mx + c$. Parameters are $\theta = \{m, c\}$.

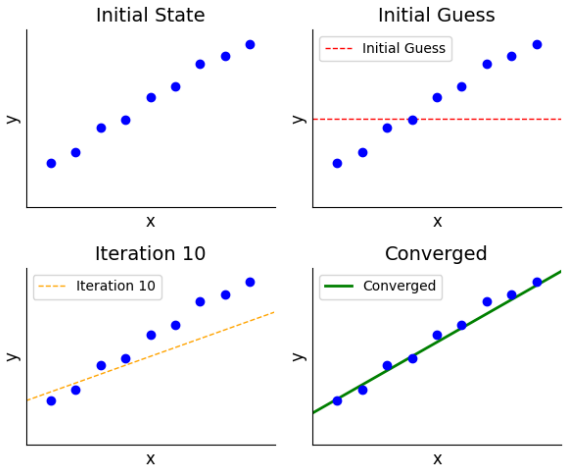
2. The Loss Function (MSE)

$$\mathcal{L}(m, c) = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$$

3. The Update Rules

$$m := m - \eta \frac{\partial \mathcal{L}}{\partial m}$$

$$c := c - \eta \frac{\partial \mathcal{L}}{\partial c}$$



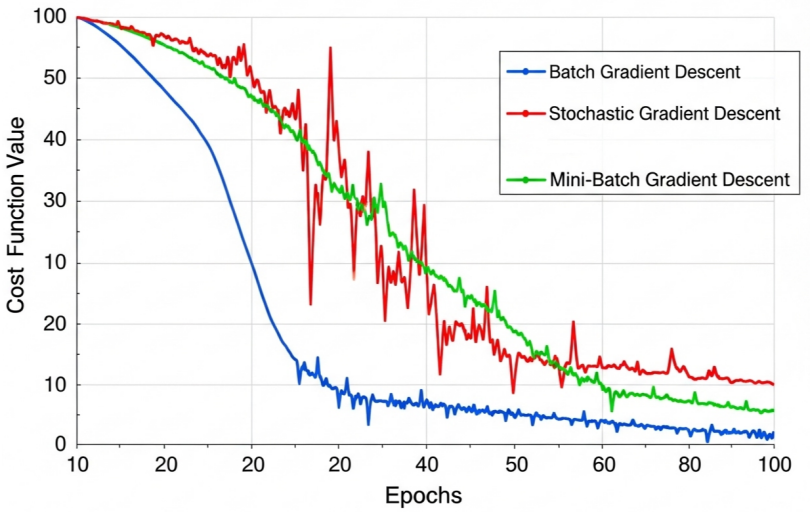
Gradient Descent: A Practical Comparison

Aspect	Batch GD	Stochastic GD (SGD)	Mini-Batch GD
Data Usage	Entire dataset	Single data point	Mini-batch of data
Update Frequency	Once per epoch	After each data point	After each mini-batch
Convergence Speed	Slow	Fast (per iteration)	Moderate
Convergence Stability	Smooth and stable	Noisy and erratic	Relatively smooth
Computational Efficiency	Inefficient for large data	Efficient	**Very Efficient (GPU)**
Memory Requirement	High	Low	Moderate

The Winner: Mini-Batch Gradient Descent

For nearly all modern deep learning applications, **Mini-Batch GD** is the standard. It offers the best compromise between the stability of Batch GD and the speed of SGD, while being perfectly suited for parallelization on GPUs.

Convergence of Gradient Descent Methods



Backpropagation - Calculating the Gradients

The Learning Loop: How Neural Networks Learn

Backpropagation's Engine: The Chain Rule

The Question

How does a small change in an early parameter (w_1) affect the final loss ($\mathcal{L}$) at the end of a long chain of computations?

The Chain Rule provides the answer by multiplying local derivatives.

Imagine a function $\mathcal{L}(y(x))$. We want to find how $\mathcal{L}$ changes with respect to x, which is $\frac{d\mathcal{L}}{dx}$.

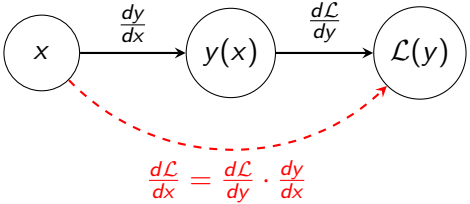
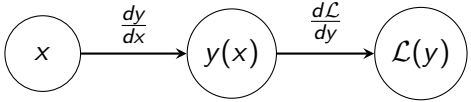


Figure: The influence of x on $\mathcal{L}$ is the product of the local influences along the path.

Scalar Case: A Simple Chain



- We know how $\mathcal{L}$ changes with respect to y ($\frac{d\mathcal{L}}{dy}$).
- We know how y changes with respect to x ($\frac{dy}{dx}$).

The chain rule lets us link them:

$$\frac{d\mathcal{L}}{dx} = \frac{d\mathcal{L}}{dy} \cdot \frac{dy}{dx}$$

Vector Case (Generalization)

For vector functions $\mathbf{y} = f(\mathbf{x})$ and $\mathcal{L} = g(\mathbf{y})$, the derivatives become Jacobian matrices, and the chain rule becomes matrix multiplication:

$$\nabla_{\mathbf{x}}\mathcal{L} = (\mathbf{J}_{\mathbf{y}}\mathcal{L})(\mathbf{J}_{\mathbf{x}}\mathbf{y})$$

This is the mathematical foundation for backpropagation in neural networks.

Visualizing Computations: The Computational Graph

Any mathematical expression can be decomposed into a graph of basic operations. This makes the flow of derivatives explicit.

Example Expression: $\mathcal{L} = (a + b) \cdot c$

We can break this down into intermediate steps: $d = a + b$ and $\mathcal{L} = d \cdot c$.

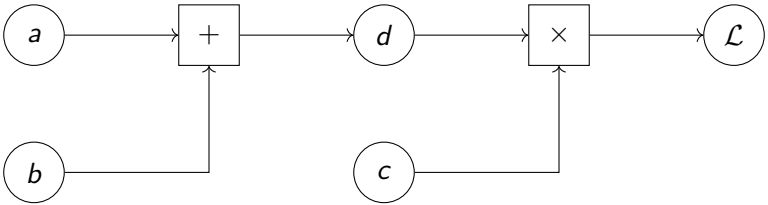


Figure: A computational graph for $\mathcal{L} = (a + b) \cdot c$. Circles are variables, boxes are operations.

Visualizing Computations: The Computational Graph

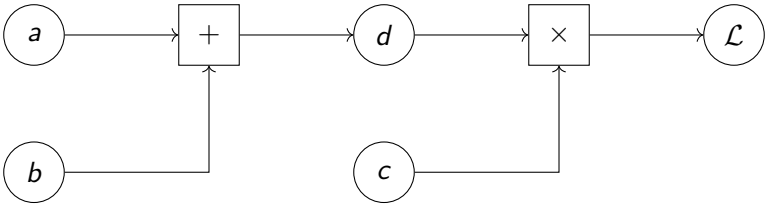


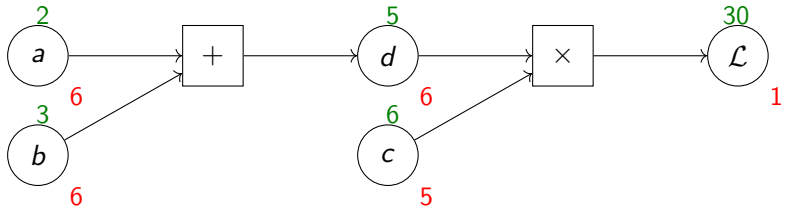
Figure: A computational graph for $\mathcal{L} = (a + b) \cdot c$. Circles are variables, boxes are operations.

The Two Passes of Automatic Differentiation

- **Forward Pass:** We start with input values (a, b, c) and compute the values of all nodes, flowing from left to right, until we get the final output $\mathcal{L}$.
- **Backward Pass (Backpropagation):** We start from the end with the gradient $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$. We then go backwards (right to left), using the chain rule at each node to compute the gradient of $\mathcal{L}$ with respect to every input of that node.

Backpropagation by Hand: A Scalar Example

Let's run backpropagation on $\mathcal{L} = (a + b) \cdot c$ with inputs $a = 2, b = 3, c = 6$.



Forward Pass (Green)

- $d = a + b = 2 + 3 = 5$
- $\mathcal{L} = d \cdot c = 5 \cdot 4 = 30$

Backward Pass (Red)

- $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$ (Start)
- $\frac{\partial \mathcal{L}}{\partial c} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \cdot \frac{\partial \mathcal{L}}{\partial c} = 1 \cdot d = 5$
- $\frac{\partial \mathcal{L}}{\partial d} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \cdot \frac{\partial \mathcal{L}}{\partial d} = 1 \cdot c = 6$
- $\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial d} \cdot \frac{\partial d}{\partial b} = 6 \cdot 1 = 6$
- $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial d} \cdot \frac{\partial d}{\partial a} = 6 \cdot 1 = 6$

Backpropagation by Hand: A Scalar Example

Summary

Node	Formula	Forward Value	Local Gradient	Final Gradient
a	-	2	-	$\frac{\partial \mathcal{L}}{\partial d}\frac{\partial d}{\partial a} = 6 \cdot 1 = 6$
b	-	3	-	$\frac{\partial \mathcal{L}}{\partial d}\frac{\partial d}{\partial b} = 6 \cdot 1 = 6$
c	-	6	-	$\frac{\partial \mathcal{L}}{\mathcal{L}}\frac{\partial \mathcal{L}}{\partial c} = 1 \cdot 5 = 5$
d	$a + b$	5	$\frac{\partial d}{\partial a} = 1, \frac{\partial d}{\partial b} = 1$	$\frac{\partial \mathcal{L}}{\partial \mathcal{L}}\frac{\partial \mathcal{L}}{\partial d} = 1 \cdot 6 = 6$
$\mathcal{L}$	$d \cdot c$	30	$\frac{\partial \mathcal{L}}{\partial d} = c = 6, \frac{\partial \mathcal{L}}{\partial c} = d = 5$	1 (by definition)

Backpropagation in a Multilayer Perceptron

The goal is to compute the gradient of the loss $\mathcal{L}$ with respect to every weight $W_{ij}^{[l]}$ and bias $b_i^{[l]}$ in the network. We do this by propagating the error signal backwards, layer by layer.

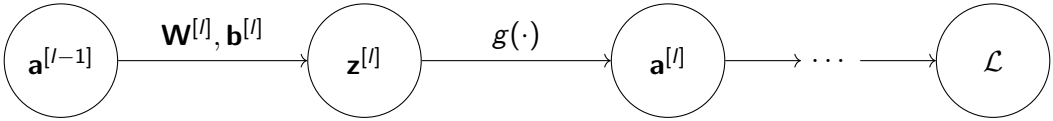


Figure: The forward pass creates a chain of dependencies. Backpropagation reverses this flow.

To update the weights $\mathbf{W}^{[l]}$, we need $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}$.

Backpropagation in a Multilayer Perceptron

The Three Key Gradient Components for a Layer l

To update the weights $\mathbf{W}^{[l]}$, we need $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}$. Using the chain rule, we decompose this:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}}_{\text{Our Goal}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}}}_{\text{1. Upstream Gradient}} \cdot \underbrace{\frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l]}}}_{\text{2. Activation Gradient}} \cdot \underbrace{\frac{\partial \mathbf{z}^{[l]}}{\partial \mathbf{W}^{[l]}}}_{\text{3. Local Gradient}}$$

1. **Upstream Gradient:** How the final loss changes with this layer's output. This is passed back from layer $l+1$.
2. **Activation Gradient:** The derivative of the activation function, $g'(\mathbf{z}^{[l]})$.
3. **Local Gradient:** How this layer's pre-activation changes with its weights.

The Weight Update Rules: The Full Derivation

Let's derive the update rules for the weights $\mathbf{W}^{[l]}$ and biases $\mathbf{b}^{[l]}$ of a hidden layer l.

Step 1: Calculate the "Error" Term $\delta^{[l]}$

We define an error term $\delta^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l]}}$, which represents how the loss changes with respect to the pre-activation of layer l.

$$\delta^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \odot \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l]}}$$

where $\odot$ is element-wise multiplication.

- The second term is just the activation derivative: $\frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l]}} = g'(\mathbf{z}^{[l]})$.
- The first term, $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}}$, is the error propagated back from the next layer, $l+1$.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} = (\mathbf{W}^{[l+1]})^T \delta^{[l+1]}$$

The Weight Update Rules: The Full Derivation

Step 1: Calculate the "Error" Term $\delta^{[l]}$

Putting it together (the core recursive step):

$$\delta^{[l]} = ((\mathbf{W}^{[l+1]})^T \delta^{[l+1]}) \odot g'(\mathbf{z}^{[l]})$$

For the output layer L, this is simpler: $\delta^{[L]} = \nabla_{\mathbf{a}^{[L]}} \mathcal{L} \odot g'(\mathbf{z}^{[L]})$.

The Weight Update Rules: The Full Derivation

Step 2: Calculate Gradients for Weights and Biases

Once we have the error $\delta^{[l]}$ for a layer, finding the gradients for its parameters is straightforward:

- **Gradient for Weights $\mathbf{W}^{[l]}$:**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{a}^{[l-1]})^T$$

- **Gradient for Biases $\mathbf{b}^{[l]}$:**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$$

These gradients are then used in an optimizer (like SGD) to update the parameters.

The XOR Problem

The Learning Loop: How Neural Networks Learn

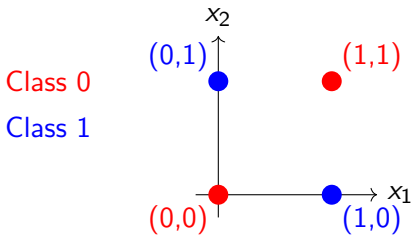
The XOR Problem: A Classic Challenge

The Exclusive OR (XOR) function is a classic problem in machine learning because it is not **linearly separable**.

XOR Truth Table

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Visualization



Key Insight

No single straight line can separate the red points (Class 0) from the blue points (Class 1). This is why simple linear models like the Perceptron fail.

XOR: Why Linear Models Fail

Let's prove mathematically that a linear model cannot solve XOR. A linear model is defined by the function:

$$f(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$

For classification, to get a class prediction $\hat{y}$.

$$\hat{y} = \sigma(f(x_1, x_2))$$

The decision boundary for a sigmoid is typically 0.5 (which corresponds to $f(x_1, x_2) = 0$).

The equation of a line in 2D, or a hyperplane in higher dimensions, which separates the two predicted classes is:

$$w_1 x_1 + w_2 x_2 + b = 0$$

XOR: Why Linear Models Fail

For the model to solve XOR, we need:

- $f(0,0) = b \approx 0$ (1)
- $f(0,1) = w_2 + b \approx 1$ (2)
- $f(1,0) = w_1 + b \approx 1$ (3)
- $f(1,1) = w_1 + w_2 + b \approx 0$ (4)

The Contradiction

From (1) and (2), we get $w_2 \approx 1$.
From (1) and (3), we get $w_1 \approx 1$.
Substituting these into (4):

$$\underbrace{w_1}_{\approx 1} + \underbrace{w_2}_{\approx 1} + \underbrace{b}_{\approx 0} \approx 0 \implies 1 + 1 + 0 \approx 0 \implies \mathbf{2 \approx 0}$$

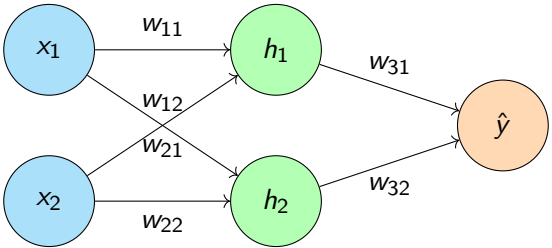
This is a contradiction, proving no linear model can solve XOR.

The Solution: A Multi-Layer Perceptron (MLP)

To solve XOR, we need a hidden layer with a **non-linear activation function**. This allows the network to create non-linear decision boundaries.

The 2-2-1 Architecture

- **Input Layer:** 2 neurons (x_1, x_2)
- **Hidden Layer:** 2 neurons (h_1, h_2) with Sigmoid activation (σ)
- **Output Layer:** 1 neuron $(\hat{y})$ with Sigmoid activation (σ)



Mathematical Formulation: Forward Pass

Hidden Layer Computations

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$
$$h_1 = \sigma(z_1)$$
$$z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$
$$h_2 = \sigma(z_2)$$

Output Layer Computation

$$z_3 = w_{31}h_1 + w_{32}h_2 + b_3$$
$$\hat{y} = \sigma(z_3)$$

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss Function (Binary Cross-Entropy)

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Forward Pass: Hand Calculation Setup

Let's walk through one forward pass for the input $\boldsymbol{x} = [1, 0]^T$ with target $y = 1$.

Specific Weights for Manual Calculation

These weights are chosen for simplicity.

- **Hidden Layer Weights (W_1):** $\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = \begin{pmatrix} 1.0 & -1.0 \\ 1.0 & -1.0 \end{pmatrix}$
- **Hidden Layer Biases (b_1):** $[b_1, b_2] = [0.0, 1.0]$
- **Output Layer Weights (W_2):** $[w_{31}, w_{32}] = [2.0, -1.0]$
- **Output Layer Bias (b_2):** $[b_3] = [-1.0]$

Goal

Calculate the network's prediction $\hat{y}$ and the resulting loss L for input $(1, 0)$.

Forward Pass: Step 1 (Hidden Layer)

Input: $x_1 = 1$, $x_2 = 0$. **Target:** $y = 1$.

Step 1: Calculate Hidden Layer Pre-Activations (z_1, z_2)

- For hidden neuron h_1:

$$\begin{aligned} z_1 &= w_{11}x_1 + w_{21}x_2 + b_1 \\ &= (1.0 \times 1) + (1.0 \times 0) + 0.0 = \mathbf{1.0} \end{aligned}$$

- For hidden neuron h_2:

$$\begin{aligned} z_2 &= w_{12}x_1 + w_{22}x_2 + b_2 \\ &= (-1.0 \times 1) + (-1.0 \times 0) + 1.0 = \mathbf{0.0} \end{aligned}$$

Forward Pass: Step 2 (Hidden Layer)

Step 2: Apply Sigmoid Activation

- $h_1 = \sigma(z_1) = \sigma(1.0) = \frac{1}{1+e^{-1}} \approx \mathbf{0.731}$
- $h_2 = \sigma(z_2) = \sigma(0.0) = \frac{1}{1+e^{0}} = \mathbf{0.500}$

Forward Pass: Step 3, 4 (Output)

Hidden Activations: $h_1 = 0.731,\ h_2 = 0.500.$ **Target:** $y = 1.$

Step 3: Calculate Output Layer Pre-Activation (z_3)

$$\begin{aligned} z_3 &= w_{31}h_1 + w_{32}h_2 + b_3 \\ &= (2.0 \times 0.731) + (-1.0 \times 0.500) + (-1.0) \\ &= 1.462 - 0.500 - 1.000 = \mathbf{-0.038} \end{aligned}$$

Step 4: Apply Sigmoid to Get Final Output ($\hat{y}$)

$$\hat{y} = \sigma(z_3) = \sigma(-0.038) = \frac{1}{1 + e^{0.038}} \approx \mathbf{0.491}$$

Forward Pass: Step 5 (Loss)

Hidden Activations: $z_3 = -0.038$, $\hat{y} = 0.491$. **Target:** $y = 1$.

Step 5: Calculate Loss ($\mathcal{L}$)

$$\begin{aligned}\mathcal{L} &= -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})] \\ &= -[1 \times \log(0.491) + (0) \times \log(1-0.491)] \\ &= -(-0.712) = \mathbf{0.712}\end{aligned}$$

Backpropagation: Mathematical Derivation

Goal

Find $\frac{\partial \mathcal{L}}{\partial w_{ij}}$ for all weights w_{ij} to update them using gradient descent:

$$w_{new} = w_{old} - \alpha \frac{\partial \mathcal{L}}{\partial w_{old}}$$

Key Derivatives

- Sigmoid derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Loss derivative (w.r.t. $\hat{y}$): $\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$

Chain Rule Structure We calculate gradients "backwards" from the output layer.

1. Calculate gradients for Output Layer weights (W_2, b_2).
2. Propagate error back to the Hidden Layer.
3. Calculate gradients for Hidden Layer weights (W_1, b_1).

Derivation: Layer-by-Layer Gradient Flow (Output)

Gradient w.r.t. Output Pre-Activation (z_3)

Using the chain rule: $\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3}$

$$\frac{\partial L}{\partial z_3} = \underbrace{\left(\frac{\hat{y} - y}{\hat{y}(1-\hat{y})}\right)}_{\partial L/\partial \hat{y}} \cdot \underbrace{(\hat{y}(1-\hat{y}))}_{\partial \hat{y}/\partial z_3}$$

$$= \hat{y} - y \quad \text{(a very convenient simplification!)}$$

Let's call this error term $\delta_3 = \hat{y} - y$.

Derivation: Layer-by-Layer Gradient Flow (Output)

Gradients for Output Layer Weights (w_{31}, w_{32}, b_3)

Apply the chain rule again:

- $\frac{\partial \mathcal{L}}{\partial w_{31}} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{31}} = \delta_3 \cdot h_1$
- $\frac{\partial \mathcal{L}}{\partial w_{32}} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{32}} = \delta_3 \cdot h_2$
- $\frac{\partial \mathcal{L}}{\partial b_3} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial b_3} = \delta_3 \cdot 1 = \delta_3$

Derivation: Layer-by-Layer Gradient Flow (Hidden)

Propagating the Error to the Hidden Layer (z_1, z_2)

We need $\frac{\partial \mathcal{L}}{\partial z_1}$ and $\frac{\partial \mathcal{L}}{\partial z_2}$.

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} = \delta_3 \cdot w_{31} \cdot \sigma'(z_1)$$
$$\frac{\partial \mathcal{L}}{\partial z_2} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial z_2} = \delta_3 \cdot w_{32} \cdot \sigma'(z_2)$$

We can define the hidden layer error terms $\delta_1 = \frac{\partial \mathcal{L}}{\partial z_1}$ and $\delta_2 = \frac{\partial \mathcal{L}}{\partial z_2}$.

Derivation: Layer-by-Layer Gradient Flow (Hidden)

Gradients for Hidden Layer Weights (w_{11}, w_{21}, etc.)

Apply the chain rule one last time:

- $\frac{\partial \mathcal{L}}{\partial w_{11}} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}} = \delta_1 \cdot x_1$
- $\frac{\partial \mathcal{L}}{\partial w_{21}} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{21}} = \delta_1 \cdot x_2$
- $\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1} = \delta_1$

(The same pattern applies for weights connected to h_2, using δ_2).

Backward Pass: Hand Calculation (Step 1 - Output)

Given from Forward Pass: $\hat{y} = 0.491, y = 1, h_1 = 0.731, h_2 = 0.500, x_1 = 1, x_2 = 0$.

Step 1: Output Layer Gradients

- Calculate the error term δ_3:

$$\delta_3 = \frac{\partial \mathcal{L}}{\partial z_3} = \hat{y} - y = 0.491 - 1 = \mathbf{-0.509}$$

- Calculate gradients for weights:

$$\frac{\partial \mathcal{L}}{\partial w_{31}} = \delta_3 \cdot h_1 = (-0.509) \times 0.731 = \mathbf{-0.372}$$
$$\frac{\partial \mathcal{L}}{\partial w_{32}} = \delta_3 \cdot h_2 = (-0.509) \times 0.500 = \mathbf{-0.255}$$

Backward Pass: Hand Calculation (Step 1 - Output)

Given from Forward Pass: $\hat{y} = 0.491, y = 1, h_1 = 0.731, h_2 = 0.500, x_1 = 1, x_2 = 0.$

Step 1 Continued: Output Layer Gradients

- Calculate gradient for bias:

$$\frac{\partial \mathcal{L}}{\partial b_3} = \delta_3 = \mathbf{-0.509}$$

Backward Pass: Hand Calculation (Step 2 - Hidden)

Given from Previous Step: $\delta_3 = -0.509, w_{31} = 2.0, w_{32} = -1.0$. **Given from Forward Pass:** $h_1 = 0.731, h_2 = 0.500, x_1 = 1, x_2 = 0$.

Step 2: Hidden Layer Gradients

- First, calculate the hidden layer error terms δ_1, δ_2:

$$\begin{aligned}
\delta_1 = \frac{\partial \mathcal{L}}{\partial z_1} &= (\delta_3 \cdot w_{31}) \cdot \sigma'(z_1) \\
&= (-0.509 \times 2.0) \cdot (h_1(1 - h_1)) \\
&= (-1.018) \cdot (0.731 \times (1 - 0.731)) = \mathbf{-0.200} \\
\delta_2 = \frac{\partial \mathcal{L}}{\partial z_2} &= (\delta_3 \cdot w_{32}) \cdot \sigma'(z_2) \\
&= (-0.509 \times -1.0) \cdot (h_2(1 - h_2)) \\
&= (0.509) \cdot (0.5 \times (1 - 0.5)) = \mathbf{0.127}
\end{aligned}$$

Backward Pass: Hand Calculation (Step 2 Cont.)

Given: $\delta_1 = -0.200, \delta_2 = 0.127, x_1 = 1, x_2 = 0.$

Step 2 Continued: Gradients for Hidden Weights & Biases

- Weights connected to h_1:

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = \delta_1 \cdot x_1 = -0.200 \times 1 = \mathbf{-0.200} \qquad \frac{\partial L}{\partial w_{21}} = \delta_1 \cdot x_2 = -0.200 \times 0 = \mathbf{0.0}$$

- Weights connected to h_2:

$$\frac{\partial \mathcal{L}}{\partial w_{12}} = \delta_2 \cdot x_1 = 0.127 \times 1 = \mathbf{0.127} \qquad \frac{\partial L}{\partial w_{22}} = \delta_2 \cdot x_2 = 0.127 \times 0 = \mathbf{0.0}$$

Backward Pass: Hand Calculation (Step 2 Cont.)

Given: $\delta_1 = -0.200, \delta_2 = 0.127, x_1 = 1, x_2 = 0.$

Step 2 Continued: Gradients for Hidden Weights & Biases

- Biases:

$$\frac{\partial \mathcal{L}}{\partial b_1} = \delta_1 = \mathbf{-0.200}$$
$$\frac{\partial \mathcal{L}}{\partial b_2} = \delta_2 = \mathbf{0.127}$$

Weight Updates

Now we use the calculated gradients to update the weights using Gradient Descent.

Update Rule

$w_{new} = w_{old} - \alpha \frac{\partial \mathcal{L}}{\partial w}$ (where α is the learning rate)

Let's use a learning rate $\alpha = 0.1$.

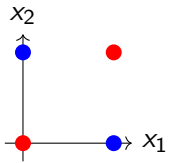
- **Output Layer Updates:**
  - $w_{31} = 2.0 - 0.1 \times (-0.372) = \mathbf{2.0372}$
  - $w_{32} = -1.0 - 0.1 \times (-0.255) = \mathbf{-0.9745}$
  - $b_3 = -1.0 - 0.1 \times (-0.509) = \mathbf{-0.9491}$
- **Hidden Layer Updates:**
  - $w_{11} = 1.0 - 0.1 \times (-0.200) = \mathbf{1.020}$
  - $w_{12} = -1.0 - 0.1 \times (0.127) = \mathbf{-1.0127}$
  - $b_1 = 0.0 - 0.1 \times (-0.200) = \mathbf{0.020}$
  - $b_2 = 1.0 - 0.1 \times (0.127) = \mathbf{0.9873}$

The weights w_{21} and w_{22} remain unchanged because their gradients were zero (since $x_2 = 0$).

Why Nonlinear Activation Succeeds: Feature Transformation

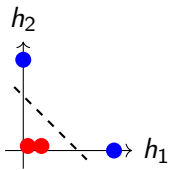
The hidden layer's crucial role is to **transform the input data** into a new representation where the problem becomes linearly separable.

Original Input Space



Not Linearly Separable

Hidden Layer Feature Space



Linearly Separable!

The "Magic" of Hidden Layers

The hidden neurons learn functions that act as feature detectors - one neuron learns "OR", another learns "NAND".